ADVERSARIAL SEARCH FOR A CHESS APPLICATION UNDER COMPUTATIONAL CONSTRAINTS

Project Proposal

ABSTRACT

This project proposes a chess-playing system to study computational effort and tactical decisions under depth and time limits. A fixed positional evaluator will support minimax in negamax form, with alpha–beta pruning and iterative deepening. Functional checks will precede a fixed-depth comparison of minimax and alpha–beta and an evaluation of tactical reference agreement under three time budgets. The planned dataset comprises 200 development and 1,000 held-out positions, equally divided between tactical puzzles and positions from recorded games. Search complexity will be analyzed separately from measured node counts, completed depth, elapsed time, and tactical accuracy. These measurements will characterize the proposed system's computational cost and tactical performance on a personal laptop.

1 PROBLEM STATEMENT

1.1 CONTEXT AND OBJECTIVES

Selecting a chess move requires anticipating the opponent's replies. Game-tree search models these continuations, while a heuristic evaluator estimates positions at the search frontier (Shannon, 1950). The number of continuations grows rapidly with depth, making exhaustive search impractical for many positions.

Existing engines already achieve strong chess play. Stockfish, for example, uses alpha–beta search with neural-network evaluation (Stockfish Developers, n.d.). The proposed system will use a fixed heuristic evaluator to support a controlled comparison of minimax and alpha–beta. The study will measure pruning savings and the effect of additional search time on completed depth and tactical move selection.

1.2 MATHEMATICAL FORMULATION

Let $\mathcal{S}$ be the set of standard-chess states. Represent a state as

$$s = (B, c, \mathcal{R}, \varepsilon, \nu, H) \in \mathcal{S}, \tag{1}$$

where B is the board, $c = c(s)$ the player to move, $\mathcal{R}$ the castling rights, ε the en-passant information, ν the move counters (half-move and full-move), and H the history needed for repetition rules. Let $\mathcal{A}(s)$ be the legal moves, $T(s, a)$ the deterministic successor, and $\mathcal{Z}$ the terminal states under the adopted game rules. A ply is one player's move.

Given a nonterminal state, a maximum depth $D \geq 1$, and a time allowance $\tau > 0$, the task is to return a legal move $\hat{a} \in \mathcal{A}(s)$ based on the adversarial continuations that can be examined. Terminal states produce a game outcome rather than a move. The underlying objective is the terminal win–draw–loss outcome; finite search approximates it through heuristic evaluation. Repetition history must be stored separately from FEN (Forsyth–Edwards Notation) or reconstructed from PGN (Portable Game Notation) (Fiekas, n.d.).

2 PROPOSED SOLUTION

2.1 POSITIONAL EVALUATION

The proposed evaluator will combine material, piece placement, pawn structure, and king safety. Material features will count pieces by type; positional features will describe square occupancy, isolated or doubled pawns, and king exposure. These quantities will be represented as differences between the two sides so that a positive score favors the player to move.

Let $f_j(s, p)$ measure feature j for player p, and let $\bar{c}(s)$ be the opponent of $c(s)$. Define the relative feature and evaluation by

$$\phi_j(s) = f_j(s, c(s)) - f_j(s, \bar{c}(s)), \qquad E_\theta(s) = \sum_{j=1}^{m} \theta_j \phi_j(s). \tag{2}$$

Here m is the number of features and $\theta \in \mathbb{R}^m$ contains their weights. Feature definitions and weights will be selected using development positions and fixed before held-out evaluation.

For terminal states, let $u(s) = -M$ when the player to move is checkmated and $u(s) = 0$ for a draw. The scale will satisfy $M > \sup_{s \notin \mathcal{Z}} |E_\theta(s)|$, ensuring that checkmate scores dominate heuristic values. Both methods will apply the same draw-adjudication policy, fixed before evaluation.

2.2 Minimax in Negamax Form

Minimax evaluates each action under the assumption that the opponent chooses its best reply. Chess is an alternating, two-player zero-sum game: an advantage for one side is a disadvantage for the other. Consequently, the identity $\min_x g(x) = -\max_x[-g(x)]$ allows both players' decisions to be expressed by the same maximization rule with a sign change.

Let $V_d(s)$ denote the depth-limited value from the perspective of the player to move, with remaining depth $d \geq 0$. The negamax recurrence is

$$V_d(s) = \begin{cases} u(s), & s \in \mathcal{Z}, \\ E_\theta(s), & s \notin \mathcal{Z},\ d = 0, \\ \max\limits_{a \in \mathcal{A}(s)} [-V_{d-1}(T(s, a))], & \text{otherwise.} \end{cases} \tag{3}$$

Terminal states are evaluated before the depth limit. At a nonterminal leaf, E_θ replaces the unknown game outcome. At an internal node, the successor value is negated because $T(s, a)$ changes the player to move. Thus each search layer chooses the best continuation for its own side.

For a nonterminal root and $d \geq 1$, the decision rule is

$$a_d \in \arg\max_{a \in \mathcal{A}(s)} [-V_{d-1}(T(s, a))]. \tag{4}$$

Ties will favor the first maximizing move in the search order. Equation 4 selects an optimal move for the depth-limited model defined by Equation 3.

2.3 Alpha–Beta Pruning

Alpha–beta will evaluate the same recurrence while avoiding branches whose remaining alternatives cannot improve the relevant ancestor's decision. A search window (α, β), with $\alpha < \beta$, records the lower threshold and the cutoff threshold for the player at that node. The best value found so far, v, starts at $-\infty$. Write $W_d(s; \alpha, \beta)$ for the value returned by windowed negamax.

Each legal action at positive remaining depth gives the candidate score

$$q_a = -W_{d-1}(T(s, a); -\beta, -\alpha). \tag{5}$$

After each call, set $v \leftarrow \max(v, q_a)$ and $\alpha \leftarrow \max(\alpha, v)$; prune the remaining actions if $\alpha \geq \beta$. The call returns v after a cutoff or after all actions are examined. The reversed signs in Equation 5 switch the perspective to the opponent. Terminal and depth-zero calls use the same u and E_θ as minimax.

A cutoff may return a bound rather than an exact internal-node value. A completed root call with an unrestricted window, however, satisfies

$$W_d(s; -\infty, +\infty) = V_d(s). \tag{6}$$

This equivalence requires identical horizons, evaluation, and terminal rules (Knuth and Moore, 1975). Move ordering affects the amount of pruning, so the controlled comparison will give both methods the same ordering and disable additional search enhancements.

2.4 Iterative Deepening and Decision Output

The proposed engine will run alpha–beta at depths $1, 2, \ldots, D$, preserving the root move from each completed iteration. A promising move from the preceding iteration may be searched first in the next iteration. This ordering supports the timed engine; it will be excluded from the separate experiment that isolates fixed-depth pruning.

Let t_d be the cumulative elapsed search time when iteration d finishes, with $t_d = +\infty$ for unfinished iterations. If at least one iteration completes within τ, return

$$d^\star = \max\{d \in \{1, \ldots, D\} : t_d \le \tau\}, \qquad \hat{a} = a_{d^\star}. \tag{7}$$

Only completed iterations will update the decision. If none finishes in time, a predetermined legal move will be returned with completed depth zero. Each query will record the move, completed depth, total visited nodes including interrupted work, and elapsed time.

Python is proposed for the search and experiments, with `python-chess` handling legal moves and FEN/PGN data (Fiekas, n.d.). The project will implement the evaluator and search procedure. A playable application will provide the human–AI interaction, while the experiments will invoke the same search procedure directly.

2.5 Data and Sampling

The dataset will contain 200 development and 1,000 held-out positions from the Lichess open database (Lichess, n.d.), equally divided between tactical puzzles and PGN game positions in each split. Puzzles provide reference moves; game positions broaden the search workload. Evaluation puzzles will be balanced across rating quartiles of a frozen pool. Each evaluation game position will be sampled from a different game using a recorded random seed.

Only legal, nonterminal positions will be included. Applying a puzzle's first listed move to its FEN yields the solver's state; the second move begins the solution. Any immediate checkmate will be accepted for mate-in-one puzzles. Development and evaluation sets will share neither source games nor duplicate positions. Source identifiers and sampling seeds will be recorded, available histories preserved, and missing pre-FEN history documented. Parameters and selection rules will be fixed before held-out evaluation.

2.6 Experimental Design

Functional verification. Tests will cover legal replies, checkmate and draw handling, castling, en passant, promotion, search interruption, and the human–AI playing workflow. Automatic draws and claimable draws will be distinguished under the selected protocol. These checks assess application behavior separately from performance or tactical accuracy.

Controlled pruning comparison. Minimax and alpha–beta will process the 500 PGN positions directly at depths $d \in \{2, 3\}$, with $R = 3$ repetitions and a safety allowance of 2000 ms per query. Both methods will use identical evaluation, move order, and terminal rules. Nodes will count entered search calls, including the root and leaves. Completed root values must agree before computational savings are compared. Other search enhancements will be disabled to isolate pruning. Development runs will check feasibility before settings are frozen; repetitions will measure timing variability.

Let $\mathcal{I}_d$ index positions where both methods complete every repetition, and $N_i^A(d)$ be the median visited-node count for method $A \in \{\mathrm{MM}, \mathrm{AB}\}$. The work ratio is

$$R_N(d) = \frac{\sum_{i \in \mathcal{I}_d} N_i^{\mathrm{MM}}(d)}{\sum_{i \in \mathcal{I}_d} N_i^{\mathrm{AB}}(d)}, \qquad \mathcal{I}_d \neq \varnothing. \tag{8}$$

A ratio above one indicates fewer nodes for alpha–beta on the completed pairs. Elapsed time and completion rates on the full set will accompany the ratio, since incomplete queries are excluded from $\mathcal{I}_d$.

Time-budgeted move selection. The proposed engine will process all 1,000 evaluation positions with $\tau \in \{100, 500, 1000\}$ ms and $R = 3$ repetitions. Queries will run sequentially on one laptop, with hardware and software versions recorded and search state reset between queries. The maximum depth D will be selected on development positions and fixed before evaluation.

For puzzle i, let $\mathcal{A}_i^{\mathrm{ref}}$ contain the reference move or all immediate mating moves when applicable, and let $\hat{a}_{i,r}(\tau)$ be the move returned in repetition r. A failed query is assigned the symbol $\perp$, which belongs to no reference set. Accuracy is

$$\mathrm{Acc}(\tau) = \frac{1}{500R} \sum_{i=1}^{500} \sum_{r=1}^{R} \mathbf{1}\{\hat{a}_{i,r}(\tau) \in \mathcal{A}_i^{\mathrm{ref}}\}. \tag{9}$$

Here $\mathbf{1}\{\cdot\}$ is one for a reference match and zero otherwise. Failed queries count as incorrect and will also be reported separately. Repeated runs do not add independent puzzles. For each budget, the report will include completed depth, total nodes, median and 95th-percentile elapsed time, and overruns. Accuracy measures agreement on the sampled puzzles, not a playing-strength rating.

3 Benefits

The system will allow deliberation time to be adjusted on a personal laptop while keeping a legal response available. Alpha–beta is expected to reduce node visits relative to unpruned minimax while preserving the completed fixed-depth value. Iterative deepening will retain a completed decision when further search must stop, making deeper search compatible with interactive play.

The experiments will quantify the cost of searching deeper and the tactical performance obtained under each time allowance. Preserved positions, fixed evaluation settings, and recorded outputs will support reproducible comparisons. Examining failures alongside reached depth and node counts will document tactical limitations and guide later changes to the search or evaluator.

4 Computational Complexity and Risks

4.1 Time Complexity

Let $b \geq 2$ be the branching factor of a full uniform game tree and d the search depth. Assuming constant processing cost per node, minimax and worst-case alpha–beta have time complexity

$$T_{\mathrm{MM}}(b, d) = O(b^d), \qquad T_{\mathrm{AB,worst}}(b, d) = O(b^d). \tag{10}$$

With ideal move ordering, alpha–beta evaluates the following minimal number of leaves (Knuth and Moore, 1975):

$$L_{\mathrm{AB,best}}(b, d) = b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1. \tag{11}$$

For fixed b, this yields best-case time $O(b^{\lceil d/2 \rceil})$, or $O(b^{d/2})$ for even depths. Actual pruning depends on move ordering.

The repeated searches of iterative deepening retain worst-case cost $O(b^d)$ because the deepest iteration dominates the geometric sum. Elapsed time also depends on move generation, evaluation, and state restoration, so it will be reported alongside node counts.

4.2 Space Complexity

With depth-first traversal and up to b candidate moves retained per active level, both methods use $O(bd)$ auxiliary search memory; the recursion stack alone is $O(d)$. This assumes incremental move application and restoration, excluding input histories and stored results.

4.3 Risks and Mitigation

Finite horizons may miss tactical continuations, while deeper search may delay responses. Iterative deepening, time checks, and a legal fallback will control deliberation; completed depth, tactical accuracy, and time overruns will be reported.

Incorrect bounds or state restoration may change search results. Agreement with unpruned minimax, legal-move checks, and restoration tests will address these risks. Search will run separately from interface handling to preserve responsiveness.

Shared source games and duplicate positions may bias evaluation; game-level separation and duplicate checks will control this risk. Repetition detection will use only recorded history, with the same draw-adjudication policy applied across methods.

5 Alternatives Considered

Unpruned minimax expands all continuations within the chosen horizon. It will serve as the reference for correctness checks and comparisons, while alpha–beta is selected as the main engine because it can avoid unnecessary branches without changing the completed fixed-depth value.

An existing engine such as Stockfish could supply strong move selection. It would delegate decisions to an external evaluation and search system, limiting control over the mechanisms being compared. Direct implementation is therefore preferred for the controlled study.

Monte-Carlo tree search allocates simulations through sampled returns and an exploration rule (Kocsis and Szepesvári, 2006). It requires a rollout or leaf-value policy and analysis of sampling variability. Alpha–beta is selected because it preserves the exact minimax value on the fixed-depth tree used in the controlled comparison.

References

Fiekas, N. (n.d.). *python-chess documentation*, version 1.11.2. `https://python-chess.readthedocs.io/en/latest/`. Accessed 30 October 2026.

Knuth, D. E. and Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326. doi:10.1016/0004-3702(75)90019-3.

Kocsis, L. and Szepesvári, C. (2006). Bandit based Monte-Carlo planning. In Fürnkranz, J., Scheffer, T., and Spiliopoulou, M., editors, *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer. doi:10.1007/11871842_29.

Lichess (n.d.). *Lichess open database*. `https://database.lichess.org/`. Accessed 30 October 2026.

Shannon, C. E. (1950). Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275. doi:10.1080/14786445008521796.

Stockfish Developers (n.d.). *Frequently asked questions*. Stockfish documentation. Accessed 30 October 2026.